Investigate, Fix or Escalate: A Rule-Constrained Reinforcement Learning Agent for IT Incident Triage under Ambiguous Evidence

Research status, November 2026

1. Why this work

When an IT problem is reported, its cause is often unclear. Staff must choose: look further, fix it, or pass it to a person. This work builds an assistant that learns that choice, with a separate rulebook check that blocks forbidden fixes.

2. Literature review: how

- **Databases:** Scopus, IEEE Xplore (open access), ACM Digital Library; 23 Oct 2026
- **Method:** PRISMA 2020; Kitchenham and Charters procedure
- **Appraisal:** eight-item checklist; synthesis around five questions
- **Limit:** one AI-assisted screener; no citation searching yet

4,399 found	73 duplicates and 4,195 removed by filters
131 screened	84 excluded
47 sought	7 not retrieved
40 read in full	6 excluded
34 included	published 2020–2026

3. What the 34 studies show

- **Hidden cause:** only 4 track what might be wrong
- **Decisions:** 17 gather information, 7 pass to a person, 1 does all three
- **Rules:** 5 use a hard block; only 3 say what happens to a blocked action
- **Measures:** quality in 31, escalation in 5, rule-keeping in 4

Research gap

No study learns one policy that investigates, fixes or escalates IT service incidents under explicit rules.

4. How the review shaped the pipeline

Review found	So this work
Unclear evidence rarely varied	→ Varies missing, wrong and overlapping evidence
Passing on seldom learned	→ Makes it a choice with a cost
Blocked actions unreported	→ Logs every block: one retry, then a person
Escalation, rule-keeping rarely measured	→ Reports four groups of measures together
Weak baselines, single runs	→ Strong baselines; several seeds

5. Dataset

- **No tickets or personal data in training:** a scenario model that IT staff fill in through a workbook
- **Pilot:** 8 causes, 11 checks, 4 symptoms, 5 rules, from one software developer
- **Planned:** about 15 causes, 20 checks, 10 rules **[TO FILL: elicitation date]**
- **Simulated tests:** 5,000 incidents never seen in training
- **Prototype:** 8 fictional tickets; 20 sample tickets kept private **[TO FILL: reader accuracy]**

6. Methodology

- **Design and evaluate,** entirely in simulation
- **Agent:** deep Q-network; **rulebook:** one retry, then a person
- **Compared with** 4 fixed, non-learning policies and 2 agents without the rulebook
- **Four experiments:** baselines, rulebook effect, choices removed, worse evidence

7. Status

- **Done:** review; pilot simulator, rulebook and agents; Experiments 1–2 (3 seeds); working prototype
- **Next:** full scenario model; 10 seeds; Experiments 3–4

How the system works

A problem report comes in → **Key facts are picked out** → **The assistant chooses a step** → **Rulebook check** → **Action is taken and recorded**

Someone reports a fault. The system's log is attached.

Each fact is copied word for word from the report or log, so a person can check it.

Look into it further, try a fix, or hand it to a person.

Steps the team's rules forbid are blocked. Blocked twice, a person takes over.

Fixes are tested first and wait for a person's approval. Every outcome is written down.

References: M. J. Page et al., *BMJ*, 2021 (PRISMA 2020). B. Kitchenham and S. Charters, EBSE-2007-01, 2007. A. R. Hevner et al., *MIS Quarterly*, 2004. M. Alshiekh et al., AAAI, 2018.